Project Level 2: Multi-Agent Analytics Assistant with Function Tools and Local MCP Server

1. Project Title

Build a Multi-Agent Analytics Assistant using CrewAI, Ollama, Streamlit, Function Tools, and a Local MCP Server

---

2. Project Difficulty Level

Level 2 — Intermediate to Advanced

This project is designed for students who already understand basic Python, Streamlit, local LLMs using Ollama, and basic CrewAI agents. In this project, students will move from a simple chat agent to a tool-using multi-agent analytics system.

---

3. Business Scenario

A company receives large volumes of business and event data from multiple sources such as CSV files, JSON logs, application events, customer transactions, and dashboard exports.

Currently, analysts manually inspect the data, write SQL queries, create KPI reports, identify data quality issues, and suggest machine learning use cases. This manual process is slow, repetitive, and difficult to scale.

The company wants to build an internal AI analytics assistant that can:

- Understand user questions
- Delegate work to specialist agents
- Use function tools for local reasoning
- Use an MCP server for reusable analytics tools
- Profile datasets
- Validate SQL queries
- Suggest KPIs
- Recommend ML problem types
- Generate a final structured answer in Streamlit

Students must build this system using only free and open-source tools.

---

4. Final System Architecture

```
User
 ↓
Streamlit Chat UI
 ↓
CrewAI Hierarchical Crew
 ↓
Supervisor Agent
 ↓
Delegates to:
  ├── Data Analyst Agent
  └── Data Scientist Agent
 ↓
Agents use:
  ├── Local Function Tools
  └── Local MCP Server Tools
 ↓
Final Answer in Streamlit
```

---

5. Agents to Build

Students must create three agents.

Agent 1: Supervisor Agent

The Supervisor Agent manages the conversation and decides how to delegate work.

Responsibilities:

- Understand the user request
- Review chat history
- Decide which specialist agent should be used
- Delegate to the Data Analyst Agent, Data Scientist Agent, or both
- Validate the final output
- Combine specialist responses into one final user-facing answer

---

Agent 2: Data Analyst Agent

The Data Analyst Agent focuses on business analysis and reporting.

Responsibilities:

- Profile datasets
- Suggest KPIs
- Validate SQL queries

- Recommend dashboards
- Identify business insights
- Explain data trends clearly

Agent 3: Data Scientist Agent

The Data Scientist Agent focuses on machine learning and advanced analytics.

Responsibilities:

- Recommend ML problem types
- Suggest feature engineering ideas
- Detect data quality issues
- Recommend model evaluation metrics
- Suggest ML pipelines
- Explain model strategy in business terms

---

6. Function Tools to Add Per Agent

Students must create local Python function tools for each agent. These tools do not require MCP. They are simple Python functions wrapped as CrewAI tools or called inside the app.

A. Supervisor Agent Function Tools

Students must implement at least 4 out of the following 5 tools.

1. `classify_user_request`

Purpose:

Classify the user request into one of the following categories:

```
analytics
data_science
sql
dashboard
data_quality
architecture
mixed
```

Example input:

```
I want to create a dashboard for revenue and churn.
```

Example output:

```
{
  "intent": "dashboard",
  "recommended_agent": "Data Analyst Agent",
  "reason": "The user is asking for dashboard and KPI design."
}
```

---

2. `create_agent_work_plan`

Purpose:

Generate a step-by-step work plan for the agents.

Example output:

```
{
  "steps": [
    "Ask Data Analyst Agent to profile the dataset.",
    "Ask Data Analyst Agent to suggest dashboard KPIs.",
    "Ask Data Scientist Agent to identify ML use cases.",
    "Combine both outputs into a final answer."
  ]
}
```

---

3. `summarize_chat_history`

Purpose:

Compress previous conversation messages into a short summary so that the context window does not become too large.

Example output:

```
The user is building a CrewAI Streamlit application with Ollama. They added a Supervisor Agent, Data Analyst Agent, and Data Scientist Agent. They now want to add MCP tools.
```

---

4. `validate_final_response_structure`

Purpose:

Check whether the final response contains required sections.

Required sections:

```
Direct Answer
Architecture
Tools Used
Step-by-Step Plan
Risks
Final Recommendation
```

---

5. `estimate_context_usage`

Purpose:

Estimate how much of the model context window is being used.

Example output:

```
{
  "estimated_input_tokens": 2300,
  "context_window": 8192,
  "usage_percent": 28.08
}
```

B. Data Analyst Agent Function Tools

Students must implement at least 4 out of the following 5 tools.

1. `profile_dataframe`

Purpose:

Profile a local CSV or DataFrame.

Output should include:

- Row count
- Column count
- Column names
- Data types
- Missing values
- Duplicate rows
- Sample records

---

2. `suggest_kpi_metrics`

Purpose:

Suggest KPIs based on dataset columns and business domain.

Example input:

```
{
  "domain": "ecommerce",
  "columns": ["order_id", "customer_id", "revenue", "order_date", "status"]
}
```

Example output:

```
{
  "recommended_kpis": [
    "Total revenue",
    "Average order value",
    "Repeat purchase rate",
    "Order cancellation rate",
    "Monthly active customers"
  ]
}
```

---

3. `generate_dashboard_layout`

Purpose:

Suggest a dashboard structure.

Output should include:

- Page name
- Chart type
- KPI card
- Filters
- Drill-down views

Example output:

```
{
  "dashboard_name": "Customer Revenue Dashboard",
  "sections": [
    "Revenue Overview",
    "Customer Segments",
    "Product Performance",
    "Churn Indicators"
```

```
  ]
}
```

---

4. `validate_sql_safety`

Purpose:

Check whether a SQL query is safe.

Rules:

- Allow only `SELECT`
- Block `DELETE`
- Block `UPDATE`
- Block `DROP`
- Block `ALTER`
- Warn if `SELECT *` is used
- Warn if no date filter is present

---

5. `explain_query_result`

Purpose:

Convert tabular output into a business explanation.

Example input:

```
{
  "metric": "monthly_revenue",
  "trend": "decreasing",
  "change_percent": -12.5
}
```

Example output:

```
Monthly revenue decreased by 12.5%. This may indicate lower customer
acquisition, reduced repeat purchases, or seasonal demand drop.
```

C. Data Scientist Agent Function Tools

Students must implement at least 4 out of the following 5 tools.

1. recommend_ml_problem_type

Purpose:

Classify the ML use case.

Possible outputs:

```
classification
regression
clustering
forecasting
anomaly_detection
recommendation
ranking
```

Example:

```
Predict whether a customer will churn.
```

Output:

```
{
  "problem_type": "classification",
  "target_variable": "churn",
  "reason": "The goal is to predict a yes/no outcome."
}
```

---

2. suggest_feature_engineering

Purpose:

Suggest features from raw event or transaction data.

Example feature ideas:

```
User activity in last 5 minutes
User activity in last 1 hour
Average transaction amount
Days since last purchase
Failed event ratio
Session duration
Number of support tickets
```

---

3. detect_ml_data_risks

Purpose:

Identify risks before model training.

Risks can include:

- Missing target column
- Class imbalance
- Data leakage
- High-cardinality columns
- Duplicate records
- Outliers
- Time-based split required

---

4. `recommend_evaluation_metrics`

Purpose:

Suggest evaluation metrics based on the ML problem type.

Example:

```
{
  "problem_type": "classification",
  "metrics": ["precision", "recall", "F1-score", "ROC-AUC", "PR-AUC"],
  "business_note": "Use recall when missing positive cases is costly."
}
```

---

5. `create_ml_pipeline_plan`

Purpose:

Create an end-to-end ML pipeline plan.

Required output sections:

```
Data ingestion
Data validation
Feature engineering
Train-test split
Model training
Model evaluation
Model registry
Batch or real-time inference
Monitoring
Retraining
```

---

7. Local MCP Server Requirement

Students must create a local MCP server named:

```
analytics_mcp_server
```

The MCP server should expose reusable open-source analytics tools. These tools should be available to agents through MCP integration.

---

8. MCP Server Tools to Implement

Students must implement at least 6 MCP tools. Advanced students can implement all 10.

MCP Tool 1: `mcp_profile_csv`

Used by:

```
Data Analyst Agent
Data Scientist Agent
```

Purpose:

Read a CSV file and return profiling information.

Open-source libraries allowed:

```
pandas
polars
duckdb
```

Expected output:

```
{
  "rows": 10000,
  "columns": 12,
  "missing_values": {},
  "duplicates": 10,
  "data_types": {},
  "sample_rows": []
}
```

MCP Tool 2: `mcp_run_duckdb_query`

Used by:

```
Data Analyst Agent
```

Purpose:

Run safe SQL queries on local CSV or Parquet files using DuckDB.

Open-source libraries allowed:

```
duckdb
pandas
polars
```

Safety requirement:

Only read-only queries should be allowed.

Blocked commands:

```
DELETE
UPDATE
DROP
ALTER
INSERT
MERGE
TRUNCATE
CREATE
```

MCP Tool 3: `mcp_validate_sql`

Used by:

```
Supervisor Agent
Data Analyst Agent
```

Purpose:

Validate SQL query safety before execution.

Open-source libraries allowed:

```
sqlglot
sqlparse
re
```

Checks:

- Query is read-only
- Query has a limit
- Query avoids `SELECT *`
- Query has a date filter when querying event data
- Query uses partition column if available

MCP Tool 4: `mcp_detect_data_quality_issues`

Used by:

```
Data Analyst Agent
Data Scientist Agent
```

Purpose:

Detect common data quality problems.

Open-source libraries allowed:

```
pandas
great_expectations
pandera
```

Checks:

- Missing values
- Duplicate rows
- Invalid data types
- Negative values in positive-only columns
- Outliers
- Constant columns
- High-cardinality columns

MCP Tool 5: `mcp_generate_kpi_catalog`

Used by:

```
Data Analyst Agent
Supervisor Agent
```

Purpose:

Generate a KPI catalog from dataset columns and business domain.

Example output:

```
{
  "domain": "ecommerce",
  "kpis": [
    {
      "name": "Conversion Rate",
      "formula": "orders / sessions",
      "grain": "daily",
      "business_use": "Tracks funnel effectiveness"
    }
  ]
}
```

MCP Tool 6: `mcp_recommend_ml_use_cases`

Used by:

```
Data Scientist Agent
Supervisor Agent
```

Purpose:

Recommend ML use cases based on dataset columns.

Example output:


```
{
  "ml_use_cases": [
    {
      "use_case": "churn prediction",
      "problem_type": "classification",
      "required_columns": ["customer_id", "activity_count", "churn_label"],
      "business_value": "Reduce customer loss"
    }
  ]
}
```

MCP Tool 7: `mcp_feature_engineering_suggestions`

Used by:

```
Data Scientist Agent
```

Purpose:

Suggest feature engineering ideas for event, customer, transaction, or time-series data.

Example output:

```
{
  "features": [
    "events_per_user_last_1_hour",
    "failed_event_ratio_last_24_hours",
    "days_since_last_purchase",
    "rolling_7_day_revenue",
    "session_count_last_30_days"
  ]
}
```

MCP Tool 8: `mcp_anomaly_detection_summary`

Used by:

```
Data Scientist Agent
Data Analyst Agent
```

Purpose:

Detect simple anomalies in numeric or time-series data.

Open-source libraries allowed:

```
pandas
scipy
scikit-learn
statsmodels
```

Methods students can use:

```
Z-score
IQR
Rolling mean deviation
Isolation Forest
Seasonal decomposition
```

MCP Tool 9: `mcp_create_data_dictionary`

Used by:

```
Data Analyst Agent
Supervisor Agent
```

Purpose:

Generate a simple data dictionary from a dataset.

Output:

```
{
  "columns": [
    {
      "name": "customer_id",
      "type": "string",
      "possible_meaning": "Unique customer identifier",
      "sample_values": ["C001", "C002"]
    }
  ]
}
```

MCP Tool 10: `mcp_generate_report_markdown`

Used by:

`Supervisor Agent`

Purpose:

Combine tool outputs into a final markdown report.

Report sections:

```
Dataset Summary
Data Quality Findings
Recommended KPIs
ML Use Cases
Risks
Next Steps
```

---

9. Open-Source Tool Mapping

Students may use the following open-source libraries:

Use Case	Recommended Library
DataFrame processing	pandas, polars
Local SQL engine	duckdb
SQL parsing	sqlglot, sqlparse
Data validation	pandera, great_expectations
ML utilities	scikit-learn

Use Case	Recommended Library
Statistics	scipy, statsmodels
Charts	matplotlib, plotly
App UI	streamlit
Agent framework	crewai
Local LLM	ollama
MCP server	mcp Python SDK

---

10. Project Folder Structure

```
level_2_multi_agent_mcp_project/
│
├── app.py
├── requirements.txt
├── README.md
│
├── config/
│   ├── agents.yaml
│   └── tasks.yaml
│
├── agents/
│   ├── supervisor_agent.py
│   ├── data_analyst_agent.py
│   └── data_scientist_agent.py
│
├── function_tools/
│   ├── supervisor_tools.py
│   ├── analyst_tools.py
│   └── scientist_tools.py
│
├── mcp_server/
│   ├── server.py
│   ├── tools/
│   │   ├── csv_profile_tools.py
│   │   ├── sql_tools.py
│   │   ├── data_quality_tools.py
│   │   ├── kpi_tools.py
│   │   ├── ml_tools.py
│   │   └── report_tools.py
│   │
│   └── sample_data/
│       ├── events_sample.csv
│       ├── transactions_sample.csv
│       └── customers_sample.csv
│
├── tests/
```

```
│   ├── test_supervisor_tools.py
│   ├── test_analyst_tools.py
│   ├── test_scientist_tools.py
│   └── test_mcp_tools.py
│
└── docs/
    ├── architecture.md
    ├── mcp_tool_catalog.md
    └── demo_script.md
```

---

11. Streamlit UI Requirements

The Streamlit app should show:

Main chat window

- User message
- Supervisor response
- Final answer

Activity timeline

Show events such as:

```
Thinking
Reasoning
Delegating
Calling function tool
Calling MCP tool
Tool result received
Final answer generated
```

Sidebar

Show:

```
Selected Ollama model
Agent list
Available function tools
Available MCP tools
Context window estimate
CrewAI usage metrics
Delegation trace
```

---

12. Expected User Demo

Students should be able to ask:

```
Analyze the events_sample.csv file. Suggest KPIs, find data quality issues,
and recommend ML use cases.
```

Expected behavior:

```
Supervisor Agent receives the request.
Supervisor Agent delegates profiling and KPI work to Data Analyst Agent.
Data Analyst Agent calls mcp_profile_csv.
Data Analyst Agent calls mcp_generate_kpi_catalog.
Data Scientist Agent calls mcp_recommend_ml_use_cases.
Data Scientist Agent calls mcp_feature_engineering_suggestions.
Supervisor Agent combines all outputs.
Streamlit displays the final structured answer.
```

---

13. Expected Final Response Format

The final answer must contain:

```
1. Direct Answer
2. Dataset Summary
3. Data Quality Findings
4. Recommended KPIs
5. Recommended Dashboard
6. ML Use Cases
7. Feature Engineering Ideas
8. Risks and Limitations
9. Next Steps
10. Agent Work Summary
```

---

14. Safety and Guardrails

Students must implement basic safety checks.

Required safety controls:

```
Block unsafe SQL queries
Do not allow arbitrary file access outside sample_data folder
Do not execute shell commands from user input
Limit file size for local uploads
Validate file extensions
Do not expose raw stack traces to the user
Put debug logs inside expandable Streamlit sections
```

---

15. Minimum Deliverables

Students must submit:

```
1. Working Streamlit app
2. Three CrewAI agents
3. agents.yaml
4. tasks.yaml
5. At least 12 local function tools total
6. At least 6 MCP server tools
7. Sample datasets
8. README.md
9. Architecture diagram
10. Demo screenshots
11. Test cases for at least 5 tools
```

---

16. Stretch Goals

Advanced students can add:

```
BigQuery metadata mock tool
Parquet file support
Real-time event simulation
Kafka-like mock event stream
Anomaly detection dashboard
Model training demo using scikit-learn
Tool call audit log
Agent memory summary
Docker support
Unit tests with pytest
```

---

17. Evaluation Rubric

Area	Marks
Multi-agent CrewAI setup	15
Supervisor delegation	10
Function tools per agent	15
MCP server implementation	20
MCP tool integration with agents	15
Streamlit UI and activity timeline	10
Safety and validation	10
Documentation and demo quality	5

Total: **100 marks**

---

18. Final Submission Prompt for Students

Students should test their final application using this prompt:

```
Analyze the events_sample.csv file. First profile the dataset, then identify data quality issues, suggest dashboard KPIs, recommend ML use cases, suggest feature engineering ideas, and provide a final implementation plan.
```

The system should use both local function tools and MCP server tools before producing the final answer.

---

19. Final Learning Outcome

After completing this project, students will understand how to:

- Build multi-agent systems with CrewAI
- Use a Supervisor Agent for delegation
- Add function tools to agents
- Build a local MCP server
- Expose analytics tools through MCP
- Integrate open-source tools into agent workflows
- Build a Streamlit UI for agent observability
- Apply safety checks for SQL and file access
- Convert raw data into business and ML recommendations